



May 2026

Prepared for
Origin ARM

Audited by
Panda
Watermelon

Origin ARM Security Review

Smart Contract Security Assessment

Contents

1	Review Summary	2
1.1	Protocol Overview	2
1.2	Audit Scope	2
1.3	Risk Assessment Framework	2
1.3.1	Severity Classification	3
1.4	Key Findings	3
1.5	Overall Assessment	4
2	Audit Overview	4
2.1	Project Information	4
2.2	Audit Timeline	4
2.3	Audit Resources	4
2.4	Critical Findings	6
2.5	High Findings	6
2.6	Medium Findings	6
2.6.1	Lido migration reuses an initializer version already consumed by a prior upgrade	6
2.6.2	OriginARM does not implement <code>_checkNoLegacyWithdrawQueue</code>	6
2.6.3	ARM implementation upgrades are blocked by unreachable <code>migrateLegacyWithdrawQueue</code> precondition	7
2.7	Low Findings	8
2.7.1	<code>AbstractARM._allocate</code> includes accrued swap fees in the liquidity buffer target	8
2.7.2	EthenaARM's storage layout is broken after refactoring the <code>unstakers</code> fixed-size array	9
2.7.3	Economically equivalent base assets can be updated inconsistently	10
2.8	Gas Savings Findings	10
2.8.1	<code>EthenaAssetAdapter.pendingUnstakerIndexes</code> is redundant and can be replaced by modular arithmetic	10
2.8.2	Branched ceiling division in <code>AbstractLidoAssetAdapter._splitAmounts</code>	12
2.9	Informational Findings	12
2.9.1	ETH and WETH donations are handled inconsistently in withdrawal adapters	12
2.9.2	<code>AbstractLidoAssetAdapter</code> approves stETH in both the constructor and initializer	13

1 Review Summary

1.1 Protocol Overview

Origin ARM is an automated redemption manager system that provides liquidity for swaps between yield-bearing base assets and liquidity assets, manages asynchronous protocol redemptions through adapters, and can allocate idle liquidity into supported ERC-4626 markets. The reviewed PR adds active-market withdrawals during swaps, replaces asset-growth performance fees with discount-based swap fees, introduces per-price liquidity limits, and moves reusable multi-base-asset logic into AbstractARM so one ARM can support multiple base assets against one liquidity asset.

1.2 Audit Scope

This audit covers 17 smart contracts totaling approximately 1,500 lines of Solidity code across 7 days of review.

src/contracts

- ├─ adapters
 - ├─ AbstractLidoAssetAdapter.sol
 - ├─ EthenaAssetAdapter.sol
 - ├─ EtherFiAssetAdapter.sol
 - ├─ OriginAssetAdapter.sol
 - ├─ StETHAssetAdapter.sol
 - ├─ WeETHAssetAdapter.sol
 - ├─ WrappedOriginAssetAdapter.sol
 - └─ WstETHAssetAdapter.sol
- ├─ AbstractARM.sol
- ├─ CapManager.sol
- ├─ EthenaARM.sol
- ├─ EtherFiARM.sol
- ├─ LidoARM.sol
- ├─ OriginARM.sol
- ├─ Ownable.sol
- └─ OwnableOperable.sol

1.3 Risk Assessment Framework

1.3.1 Severity Classification

Severity	Description	Potential Impact
Critical	Immediate threat to user funds or protocol integrity	Direct loss of funds, protocol compromise
High	Significant security risk requiring urgent attention	Potential fund loss, major functionality disruption
Medium	Important issue that should be addressed	Limited fund risk, functionality concerns
Low	Minor issue with minimal impact	Best practice violations, minor inefficiencies
Undetermined	Findings whose impact could not be fully assessed within the time constraints of the engagement. These issues may range from low to critical severity, and although their exact consequences remain uncertain, they present a sufficient potential risk to warrant attention and remediation.	Varies based on actual severity
Gas	Findings that can improve the gas efficiency of the contracts.	Increased transaction costs
Informational	Code quality and best practice recommendations	Reduced maintainability and readability

Table 1: severity classification

1.4 Key Findings

Breakdown of Finding Impacts

Impact Level	Count
■ Critical	0
■ High	0
■ Medium	3
■ Low	3
■ Informational	2

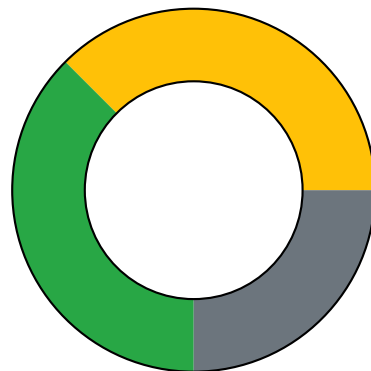


Figure 1: Distribution of security findings by impact level

1.5 Overall Assessment

The review found no critical or high severity issues. The main risks were concentrated around upgrade-time storage migration, legacy withdrawal queue handling, and multi-asset accounting edge cases. All findings were fixed during the engagement, improving the safety of the upgrade and adapter architecture.

2 Audit Overview

2.1 Project Information

Protocol Name: Origin ARM

Repository: <https://github.com/OriginProtocol/arm-oeth>

Commit Hash: dd55fae5a9783f9e5b43dad96eaa292c8f473c32

Commit URL:

<https://github.com/OriginProtocol/arm-oeth/tree/dd55fae5a9783f9e5b43dad96eaa292c8f473c32>

2.2 Audit Timeline

The audit was conducted from May 18 to 26, 2026.

2.3 Audit Resources

- Code repository and pull request #208
- Deployment scripts, tests, and protocol diagrams included in the repository

Category	Mark	Description
Access Control	Good	The system uses owner/operator separation and restricted adapter entrypoints.
Mathematics	Good	Core accounting is non-trivial due to share pricing, queued LP redemptions, adapter pending values, swap fees, and active market allocation. Several edge cases were identified around migration, valuation, and buffer accounting.
Complexity	Good	The codebase combines upgradeable storage layout constraints, multiple asset adapters, asynchronous withdrawal queues, and ERC-4626 market allocation, creating meaningful integration complexity.
Libraries	Excellent	The implementation relies on established OpenZeppelin components and standard ERC interfaces, with protocol-specific integrations isolated mostly in adapters.
Decentralization	Good	Privileged operators and owners retain important control over upgrades, prices, supported assets, market allocation, and emergency pause behavior. No unexpected centralization issue was identified beyond these intended controls.
Documentation	Good	NatSpec and inline comments generally explain the key accounting flows and upgrade migration intent. Some adapter and edge-case behavior required clarification through findings.
Testing and verification	Good	The findings indicate that upgrade-state migration and certain multi-asset accounting edge cases needed stronger regression coverage.

Table 2: Code Evaluation Matrix

2.4 Critical Findings

None.

2.5 High Findings

None.

2.6 Medium Findings

2.6.1 Lido migration reuses an initializer version already consumed by a prior upgrade

Technical Details

`migrateLegacyWithdrawQueue()` is intended to run during the new Lido upgrade via `upgradeToAndCall`, but it is guarded by `reinitializer(2)`:

```
1 function migrateLegacyWithdrawQueue() external onlyOwner reinitializer(2)
```

The previous Lido withdrawal-registration upgrade already used initializer version 2. In the pre-PR implementation `registerLidoWithdrawalRequests()` was also `reinitializer(2)`.

Impact

Medium. The Lido upgrade can be blocked.

Recommendation

Use an unused initializer version, such as `reinitializer(3)`.

Developer Response

Good find. Fixed in PR <https://github.com/OriginProtocol/arm-oeth/pull/250>

2.6.2 OriginARM does not implement `_checkNoLegacyWithdrawQueue`

Technical Details

`AbstractARM.migrateLegacyWithdrawQueue` is intended to be called via `upgradeToAndCall` during an implementation upgrade and is responsible for clearing the legacy packed asset queue counter slot. Before doing so, it invokes the virtual `_checkNoLegacyWithdrawQueue` hook to allow protocol-specific implementations to revert if any legacy withdrawal request is still outstanding.

`LidoARM`, `EtherFiARM`, and `EthenaARM` all override `_checkNoLegacyWithdrawQueue` to enforce that their respective legacy withdrawal queue tracker is zero before the migration is allowed to proceed.

`OriginARM` retains an analogous legacy tracker, `_deprecatedVaultWithdrawalAmount`, exposed via the `vaultWithdrawalAmount` view. However, `OriginARM` does not override `_checkNoLegacyWithdrawQueue`, so the default empty implementation defined in `AbstractARM` is used. As a result, `migrateLegacyWithdrawQueue` does not verify that the legacy Origin vault withdrawal queue is empty before completing the upgrade.

Because the post-upgrade implementation moves vault withdrawal state to the Origin asset adapter, any legacy withdrawal request still tracked under `_deprecatedVaultWithdrawalAmount` at the time of the upgrade would no longer be claimable by the new ARM, stranding the associated assets.

Impact

Medium. The issue is contingent on the upgrade being executed while vault withdrawals are still outstanding, but the missing override removes the on-chain safety net relied on by every other ARM implementation. Any resulting stranding of funds is temporary, as a subsequent implementation upgrade exposing a rescue path for the legacy withdrawal queue would allow the affected assets to be recovered.

Recommendation

Override `_checkNoLegacyWithdrawQueue` in `OriginARM` to revert if any legacy vault withdrawal is still outstanding:

```
1 function _checkNoLegacyWithdrawQueue() internal view override {  
2     require(_deprecatedVaultWithdrawalAmount == 0, "OriginARM: withdrawals pending");  
3 }
```

Developer Response

Fixed in PR [Adds legacy withdrawal queue safety checks and selector documentation](#)

2.6.3 ARM implementation upgrades are blocked by unreachable `migrateLegacyWithdrawQueue` precondition

Technical Details

The new `AbstractARM` recycles storage slot 53. In the legacy layout the slot packs `withdrawsQueued` (low 128 bits) and `withdrawsClaimed` (high 128 bits); in the new layout the same slot holds `reservedWithdrawLiquidity`, the running total of liquidity reserved against outstanding LP redemption requests. Because the two interpretations are mutually incompatible, any non-zero residue from the legacy packing would be read by the new code as a multi-tens-of-thousands of ether reserve and corrupt every accounting path that consumes it. To make the recycle safe, the upgrade ships `AbstractARM.migrateLegacyWithdrawQueue`, a `reinitializer(2)` that zeroes slot 53. The reset is gated by:

```
1 if (legacyQueued != legacyClaimed) revert LegacyWithdrawalsPending();
```

The production scripts ([028](#), [029](#), [030](#)) invoke the reinitializer atomically via `upgradeToAndCall`, so the guard must hold in the same block the new implementation is wired in.

Under standard ARM operation the guard does not hold. The contract exists to service async LP redemptions behind a 10-minute claim delay, so at nearly every block there is at least one matured-but-unclaimed or still-maturing request in flight, which keeps `withdrawsQueued > withdrawsClaimed`. The legacy implementation exposes no way to suspend `requestRedeem` either, so the operator cannot freeze the queue before the upgrade lands.

Live state on the Lido ARM proxy `0x85B78AcA6Deae198fBF201c82DAF6Ca21942acc6` at the time of writing illustrates the misalignment:

withdrawsQueued = 27,951,253,590,854,368,215,523
withdrawsClaimed = 27,951,098,163,810,627,874,485
delta $\approx$ 0.1554 WETH outstanding across 354 lifetime requests

Impact

Medium. The new implementation cannot be deployed against any live ARM with outstanding legacy requests.

Recommendation

Consider using a new storage slot for `reservedWithdrawLiquidity` instead of recycling slot 53 and modifying `AbstractARM.claimRedeem` such that it is able to handle both pre and post migration redemption requests: the method should use the old storage variables or the new one based on when the redemption request was submitted.

Developer Response

Fixed in PR [Support claiming legacy redeem requests](#)

2.7 Low Findings

2.7.1 `AbstractARM._allocate` includes accrued swap fees in the liquidity buffer target

Technical Details

`AbstractARM._allocate()` determines how much of the ARM's liquidity should remain on-hand versus be deposited into the active lending market. It uses `_availableAssets()` as the basis for this calculation which returns the gross asset value of the ARM, including the `feesAccrued` portion owed to the fee collector.

Because the buffer target is computed against the gross asset value instead of the LP-attributable value `_availableAssets() - feesAccrued`, `targetArmLiquidity` is smaller than it should be by approximately `feesAccrued * (1 - armBuffer / 1e18)`. The resulting `targetLiquidityDelta` therefore over-deposits into the active market by that amount on every call to `allocate()`, effectively allocating a fraction of the accrued swap fees to the lending market.

In addition to the conceptual mis-allocation, this can block fee collection: if the ARM's on-hand liquidity asset balance has been reduced below `feesAccrued + reservedWithdrawLiquidity` by allocation, `collectFees()` will revert until an operator manually pulls liquidity back from the lending market.

Impact

Low. A portion of accrued swap fees is consistently allocated to the active lending market.

Recommendation

Exclude `feesAccrued` from the buffer basis so the allocation calculation operates on LP-attributable assets only and keeps the accrued fees on-hand.

Developer Response

Acknowledged, but this is intended/acceptable behaviour. `feesAccrued` are excluded from LP `totalAssets()` accounting, but until collected they remain assets held by the ARM and may be allocated to the active market along with other liquidity. `collectFees()` intentionally requires sufficient on-hand liquidity, so fee collection may need to wait for liquidity to be available or for an operator to pull funds back from the market. This does not cause loss of funds or incorrect LP accounting.

2.7.2 EthenaARM's storage layout is broken after refactoring the `unstakers` fixed-size array

Technical Details

The previous version of EthenaARM declared, in order, the following state variables:

```
1 uint256 public liquidityAmountInCooldown;           // slot N
2 address[MAX_UNSTAKERS] public unstakers;           // slots N+1 .. N+42 (MAX_UNSTAKERS
  = 42)
3 uint8 public nextUnstakerIndex;                     // slot N+43 (packed)
4 uint32 public lastRequestTimestamp;                 // slot N+43 (packed with the above
  )
```

The new version of the contract refactors these into deprecated placeholders:

```
1 uint256 internal _deprecatedLiquidityAmountInCooldown; // slot N
2 uint256 internal _deprecatedUnstakers;                 // slot N+1
3 uint8 internal _deprecatedNextUnstakerIndex;           // slot N+2 (packed)
4 uint32 internal _deprecatedLastRequestTimestamp;       // slot N+2 (packed)
```

The fixed-size `address[42]` array used to occupy 42 contiguous storage slots starting at its declaration slot, but the replacement `_deprecatedUnstakers` only reserves a single slot. As a result, every storage slot from `N+2` onwards in the new layout maps to a slot that previously belonged to `unstakers[1]` through `unstakers[41]` in the legacy contract:

- `_deprecatedNextUnstakerIndex` and `_deprecatedLastRequestTimestamp` now share the slot that used to hold `unstakers[1]`, an `address` set by `setUnstakers`. If a legacy deployment had populated the `unstakers` array prior to the upgrade, the post-upgrade contract would read non-zero garbage out of these two variables. While neither variable is referenced after the upgrade, the slot itself is occupied by stale data.
- The remaining 40 slots that previously held `unstakers[2]` through `unstakers[41]` are now unallocated by the new layout. The contract does not declare a `__gap` array to reserve them.

Because the contract is deployed behind an upgradeable proxy, any state variable appended to EthenaARM in a future upgrade will be assigned to slots that potentially hold leftover non-zero address values from the previous `unstakers` array. These new variables would therefore initialize to corrupted values rather than zero, with the exact corruption depending on which legacy addresses were stored at each slot.

Impact

Low. The current upgrade does not exercise the affected slots, so no immediate corruption is observed. However, the storage layout is left in a state where any future upgrade that introduces new state in EthenaARM will silently collide with stale legacy data.

Recommendation

Preserve the legacy storage layout faithfully by reintroducing the 42-element fixed-size array placeholder, keeping the `_deprecated` naming convention used for the other slots:

```
1      /// @dev Deprecated unstaker helper array retained for storage layout
      compatibility.
2  -   uint256 internal _deprecatedUnstakers;
3  +   address[42] internal _deprecatedUnstakers;
```

Developer Response

Fixed in PR [Restore slot storage of Ethena ARM](#)

2.7.3 Economically equivalent base assets can be updated inconsistently

Technical Details

`stETH` and `wstETH` are economically equivalent base assets, but they are configured independently in `baseAssetConfigs`. The current `setPrices()` and `setCrossPrice()` functions update only one base asset at a time, so a bot or operator can temporarily update one side while leaving the other side stale.

If the `wstETH` sell price is updated but the `stETH` buy price is stale, a trader can swap `WETH -> wstETH` through the ARM, unwrap through Lido, and then swap `stETH -> WETH` through the ARM. This is profitable when `STETH.buyPrice > WSTETH.sellPrice`. The reverse route is profitable when `WSTETH.buyPrice > STETH.sellPrice`.

Impact

Low. Traders can arbitrage stale cross-asset pricing if the mismatch exceeds the ARM spread.

Recommendation

Add a batch price update function for related base assets. Given that contract size is an issue, a different solution is to turn the bot EOA into a smart contract using 7702, so the two calls can be grouped together.

Developer Response

Ack, won't change. Base asset cross prices and buy/sell prices can be batched in governance proposal on Ethereum or a Safe tx if not on Ethereum.

2.8 Gas Savings Findings

2.8.1 `EthenaAssetAdapter.pendingUnstakerIndexes` is redundant and can be replaced by modular arithmetic

Technical Details

The `EthenaAssetAdapter` contract tracks the order in which unstakers are used to service redemption requests via the storage array `pendingUnstakerIndexes`. The only push site is `requestRedeem`, which pushes the current `nextUnstakerIndex` and immediately increments it modulo `MAX_UNSTAKERS`. `nextUnstakerIndex` is never written elsewhere in the contract, and the storage default at deployment is `0`.

As a consequence, the following invariant holds for every valid array position `k`:

```
pendingUnstakerIndexes[k] == k % MAX_UNSTAKERS
```

The array is therefore a stored, deterministic repetition of the sequence `[0, 41]`, with each entry being directly derivable from its position and `MAX_UNSTAKERS`.

Impact

Gas optimization.

Recommendation

Replace the `pendingUnstakerIndexes` array with a `totalRequests` counter and derive index values via modular arithmetic:

```
1 -   uint8[] internal pendingUnstakerIndexes;
2 +   uint256 internal totalRequests;
3     uint256 internal nextPendingIndex;

1   function requestRedeem(uint256 shares) external onlyARM nonZeroShares(
    shares) ... {
2       ...
3 -     pendingUnstakerIndexes.push(nextUnstakerIndex);
4 +     totalRequests++;
5       nextUnstakerIndex = uint8((nextUnstakerIndex + 1) % MAX_UNSTAKERS);
6       ...
7   }

9   function redeem(uint256 shares) external onlyARM nonZeroShares(shares) ...
    {
10 -     uint256 length = pendingUnstakerIndexes.length;
11 +     uint256 length = totalRequests;
12     uint256 cursor = nextPendingIndex;
13     uint256 claimCount;

15     while (cursor + claimCount < length && sharesClaimed < shares) {
16 -     address unstaker = unstakers[pendingUnstakerIndexes[cursor +
    claimCount]];
17 +     address unstaker = unstakers[(cursor + claimCount) % MAX_UNSTAKERS
    ];
18     ...
19     }
20     ...
21     for (uint256 i = 0; i < claimCount; ++i) {
22 -     address unstaker = unstakers[pendingUnstakerIndexes[cursor + i]];
23 +     address unstaker = unstakers[(cursor + i) % MAX_UNSTAKERS];
24     ...
25     }
26 }
```

The external view methods `pendingUnstakerIndexesLength()` and `pendingUnstakerIndex(uint256)` should be preserved by deriving their return values from `totalRequests` and the modular expression respectively.

Developer Response

Fixed in PR [Optimize Ethena unstaker queue tracking](#)

2.8.2 Branched ceiling division in `AbstractLidoAssetAdapter._splitAmounts`

Technical Details

`AbstractLidoAssetAdapter._splitAmounts` computes the number of `MAX_WITHDRAWAL_AMOUNT` chunks required to fulfill a given withdrawal `amount`. The current implementation computes the ceiling of the division `amount / MAX_WITHDRAWAL_AMOUNT` using a branched pattern that performs both a division and a modulo, then conditionally increments the result. The equivalent ceiling division can be expressed branchlessly with a single division, replacing the modulo and conditional with cheaper arithmetic operations.

Impact

Gas optimization.

Recommendation

Replace the branched ceiling division with its branchless equivalent:

```
1 @@ -143,8 +143,7 @@ abstract contract AbstractLidoAssetAdapter is Initializable,
   IAssetAdapter {
2     function _splitAmounts(uint256 amount) internal pure returns (uint256[]
       memory amounts) {
3         uint256 chunkCount = amount / MAX_WITHDRAWAL_AMOUNT;
4         if (amount % MAX_WITHDRAWAL_AMOUNT != 0) chunkCount++;
5         uint256 chunkCount = (amount + MAX_WITHDRAWAL_AMOUNT - 1) /
           MAX_WITHDRAWAL_AMOUNT;
7         amounts = new uint256[](chunkCount);
```

Developer Response

The gas saving here is pretty minimal. We'll keep it as is as I think the existing code is more readable.

2.9 Informational Findings

2.9.1 ETH and WETH donations are handled inconsistently in withdrawal adapters

Technical Details

Adapters generally snapshot the output ERC20 balance before claiming and only transfer the balance delta to the ARM. This keeps `assetsReceived` limited to assets produced by the claim and excludes pre-existing ERC20 donations.

`WeETHAssetAdapter`, `EtherFiAssetAdapter`, and `AbstractLidoAssetAdapter` follow this pattern for WETH by snapshotting `wethBefore`.

However, they wrap the full native ETH balance after claiming:

```
1 uint256 ethBalance = address(this).balance;
2 if (ethBalance > 0) weth.deposit{value: ethBalance}();
```

This creates a native-ETH exception to the otherwise consistent delta-accounting pattern: pre-existing donated ETH is swept to the ARM and included in `assetsReceived`, while pre-existing donated WETH is excluded.

Impact

Informational.

Recommendation

Make donation handling explicit and consistent. Either snapshot ETH too so only claim proceeds are transferred, or intentionally sweep both ETH and WETH to the ARM and document that behavior.

Developer Response

Fixed in PR [Sweep donated ETH and WETH from claim adapters](#)

2.9.2 AbstractLidoAssetAdapter approves stETH in both the constructor and initializer

`AbstractLidoAssetAdapter` grants the Lido withdrawal queue a max stETH allowance in both the constructor and `initialize()`.

Technical Details

The adapter approves stETH to the Lido withdrawal queue in the constructor:

```
1 IERC20(_steth).approve(_lidoWithdrawalQueue, type(uint256).max);
```

It then repeats the same approval in `initialize()`:

```
1 IERC20(address(steth)).approve(address(lidoWithdrawalQueue), type(uint256).max);
```

For proxied deployments, the constructor approval is made by the implementation contract address, not by the proxy. The proxy still needs `initialize()` to be called so the proxy itself grants the withdrawal queue allowance.

For direct deployments, the constructor approval is already sufficient, and calling `initialize()` repeats the same approval from the same contract address.

Impact

Informational.

Recommendation

Remove the `approve` from the constructor.

Developer Response

Fixed in PR [Fix token approval in adapters](#)



Origin ARM Security Review

Completed 2026-05-26